Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 2.5

Obtained Marks: _____

Artificial Neural Networks

(Lab)

Task # 02

Submitted To: Mr. Hassaan Mujtaba

Student Name: Taimur sultan

Registration Number: 24108145

Dataset 1:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
# 1. Load the data
df = pd.read_csv("Data_final.csv")
print(df.head())
# 2. Choose input columns (X) and target column (y)
X = df[["Numerical Aptitude", "Spatial Aptitude", "Perceptual Aptitude",
        "Abstract Reasoning", "Verbal Reasoning"]]
y = df["Career"]            # categorical target -> classification
# 3. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 4. Train the Logistic Regression model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
# 5. Check how good the model is
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, zero_division=0))
# 6. Prediction function (used by both Gradio and Flask)
def predict(numerical, spatial, perceptual, abstract, verbal):
    data = pd.DataFrame([[numerical, spatial, perceptual, abstract, verbal]],
                        columns=X.columns)
    return model.predict(data)[0]
# ===== 7. Gradio Interface =====
import gradio as gr

app_gradio = gr.Interface(
    fn=predict,
    inputs=[
        gr.Number(label="Numerical Aptitude"),
        gr.Number(label="Spatial Aptitude"),
        gr.Number(label="Perceptual Aptitude"),
        gr.Number(label="Abstract Reasoning"),
        gr.Number(label="Verbal Reasoning"),
    ],
    outputs=gr.Textbox(label="Predicted Career"),
    title="Career Predictor (Logistic Regression)",
)

app_gradio.launch()
```

```
# ===== 8. Flask Interface =====
from flask import Flask, request, render_template_string

app = Flask(__name__)

html = """
<h2>Career Predictor (Logistic Regression)</h2>
<form method="post">
    Numerical Aptitude: <input name="numerical" required><br><br>
    Spatial Aptitude: <input name="spatial" required><br><br>
    Perceptual Aptitude: <input name="perceptual" required><br><br>
    Abstract Reasoning: <input name="abstract" required><br><br>
    Verbal Reasoning: <input name="verbal" required><br><br>
    <button type="submit">Predict</button>
</form>
<h3>{{ result }}</h3>
"""

@app.route("/", methods=["GET", "POST"])
def home():
    result = ""
    if request.method == "POST":
        numerical = float(request.form["numerical"])
        spatial = float(request.form["spatial"])
        perceptual = float(request.form["perceptual"])
        abstract = float(request.form["abstract"])
        verbal = float(request.form["verbal"])
        result = "Predicted Career: " + str(
            predict(numerical, spatial, perceptual, abstract, verbal))
    return render_template_string(html, result=result)

app.run(port=5000)
```

Output :

	O_score	C_score	E_score	A_score	N_score	Numerical Aptitude	\
0	5.45	8.67	3.45	5.34	4.23	9.23	
1	8.78	5.67	4.56	6.45	4.23	5.12	
2	6.12	6.78	9.34	7.56	5.01	6.23	
3	9.12	8.78	4.23	5.67	4.56	7.89	
4	6.45	7.56	5.67	9.12	4.23	5.34	

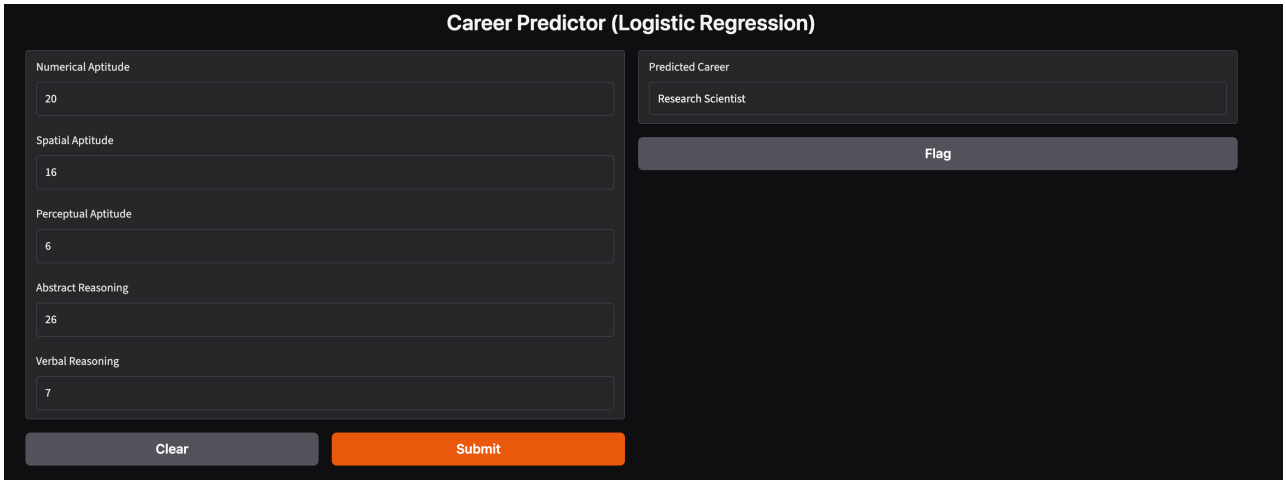
	Spatial Aptitude	Perceptual Aptitude	Abstract Reasoning	\
0	4.56	6.78	7.89	
1	8.45	7.89	6.34	
2	4.23	6.45	6.67	
3	5.34	6.45	9.34	
4	4.01	6.23	5.78	

	Verbal Reasoning	Career
0	6.12	Accountant
1	6.01	Graphic Designer
2	8.45	Salesperson
3	7.67	Research Scientist
4	8.67	Teacher

```
/opt/anaconda3/lib/python3.14/site-packages/sklearn/linear_model/_logistic.py:1469: UserWarning: The number of unique classes is greater t
han 50% of the number of samples. `y` could represent a regression problem, not a classification problem.
  check_classification_targets(y)
Accuracy: 0.0
```

	precision	recall	f1-score	support
Accountant	0.00	0.00	0.00	1.0
Advertising Executive	0.00	0.00	0.00	0.0
Artist	0.00	0.00	0.00	1.0
Biologist	0.00	0.00	0.00	1.0
Biomedical Engineer	0.00	0.00	0.00	1.0
Biotechnologist	0.00	0.00	0.00	0.0
Data Analyst	0.00	0.00	0.00	0.0
Database Analyst	0.00	0.00	0.00	1.0
Diplomat	0.00	0.00	0.00	1.0
Electrical Engineer	0.00	0.00	0.00	0.0
Electronics Design Engineer	0.00	0.00	0.00	1.0
Environmental Scientist	0.00	0.00	0.00	0.0
Fashion Designer	0.00	0.00	0.00	0.0
Financial Advisor	0.00	0.00	0.00	1.0
Financial Analyst	0.00	0.00	0.00	0.0
Financial Planner	0.00	0.00	0.00	1.0
Foreign Service Officer	0.00	0.00	0.00	0.0
Forensic Scientist	0.00	0.00	0.00	1.0
Forestry Technician	0.00	0.00	0.00	0.0
Geologist	0.00	0.00	0.00	1.0
HR Recruiter	0.00	0.00	0.00	1.0
IT Project Manager	0.00	0.00	0.00	1.0
Insurance Underwriter	0.00	0.00	0.00	1.0
Investment Banker	0.00	0.00	0.00	1.0
Journalist	0.00	0.00	0.00	0.0
Marine Biologist	0.00	0.00	0.00	0.0
Marketing Manager	0.00	0.00	0.00	1.0
Psychologist	0.00	0.00	0.00	0.0
Public Health Analyst	0.00	0.00	0.00	0.0
Rehabilitation Counselor	0.00	0.00	0.00	1.0
Salesperson	0.00	0.00	0.00	0.0
Social Worker	0.00	0.00	0.00	1.0
Software Developer	0.00	0.00	0.00	0.0
Tax Collector	0.00	0.00	0.00	1.0
Teacher	0.00	0.00	0.00	1.0
Technical Writer	0.00	0.00	0.00	0.0
Wildlife Conservationist	0.00	0.00	0.00	1.0
accuracy			0.00	21.0
macro avg	0.00	0.00	0.00	21.0
weighted avg	0.00	0.00	0.00	21.0

Flask interface:



Dataset 2:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# 1. Load the data
df = pd.read_csv("diabetes_prediction_dataset.csv")
print(df.head())

# 2. Choose input columns (X) and target column (y)
X = df[["age", "bmi", "HbA1c_level", "blood_glucose_level"]]
y = df["diabetes"]          # 0 = No Diabetes, 1 = Diabetes

# 3. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Train the Logistic Regression model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# 5. Check how good the model is
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))


# 6. Prediction function (used by both Gradio and Flask)
def predict(age, bmi, hba1c, glucose):
    data = pd.DataFrame([[age, bmi, hba1c, glucose]], columns=X.columns)
    result = model.predict(data)[0]
    prob = model.predict_proba(data)[0][1]
    label = "Diabetic" if result == 1 else "Not Diabetic"
    return f"{label} (probability of diabetes: {round(prob * 100, 2)}%)"


# ===== 7. Gradio Interface =====
import gradio as gr

app_gradio = gr.Interface(
    fn=predict,
    inputs=[
        gr.Number(label="Age"),
        gr.Number(label="BMI"),
        gr.Number(label="HbA1c Level"),
        gr.Number(label="Blood Glucose Level"),
```

```
# ===== 7. Gradio Interface =====
import gradio as gr

app_gradio = gr.Interface(
    fn=predict,
    inputs=[
        gr.Number(label="Age"),
        gr.Number(label="BMI"),
        gr.Number(label="HbA1c Level"),
        gr.Number(label="Blood Glucose Level"),
    ],
    outputs=gr.Textbox(label="Diabetes Prediction"),
    title="Diabetes Predictor (Logistic Regression)",
)

app_gradio.launch()
# ===== 8. Flask Interface =====
from flask import Flask, request, render_template_string

app = Flask(__name__)

html = """
<h2>Diabetes Predictor (Logistic Regression)</h2>
<form method="post">
    Age: <input name="age" required><br><br>
    BMI: <input name="bmi" required><br><br>
    HbA1c Level: <input name="hba1c" required><br><br>
    Blood Glucose Level: <input name="glucose" required><br><br>
    <button type="submit">Predict</button>
</form>
<h3>{{ result }}</h3>
"""

@app.route("/", methods=["GET", "POST"])
def home():
    result = ""
    if request.method == "POST":
        age = float(request.form["age"])
        bmi = float(request.form["bmi"])
        hba1c = float(request.form["hba1c"])
        glucose = float(request.form["glucose"])
        result = "Prediction: " + predict(age, bmi, hba1c, glucose)
    return render_template_string(html, result=result)

app.run(port=5001)
```

Output :

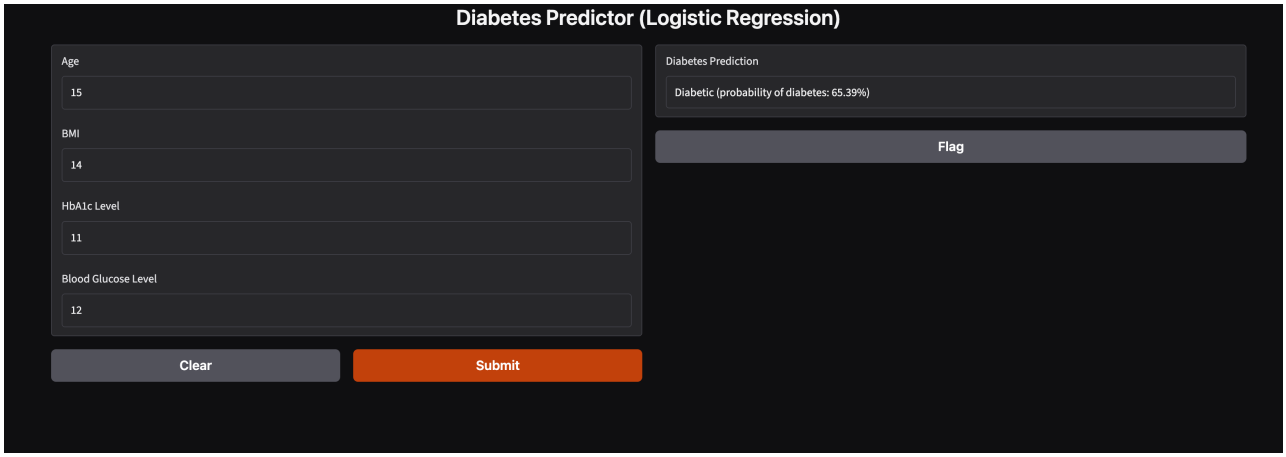
```
    gender   age  hypertension  heart_disease smoking_history    bmi  \
0   Female  80.0             0              1           never  25.19
1   Female  54.0             0              0         No Info  27.32
2     Male  28.0             0              0           never  27.32
3   Female  36.0             0              0         current  23.45
4     Male  76.0             1              1         current  20.14

    HbA1c_level  blood_glucose_level  diabetes
0           6.6                  140         0
1           6.6                   80         0
2           5.7                  158         0
3           5.0                  155         0
4           4.8                  155         0
Accuracy: 0.9586
              precision    recall  f1-score   support

           0       0.96      0.99      0.98     18292
           1       0.88      0.60      0.71      1708

    accuracy                           0.96     20000
   macro avg       0.92      0.80      0.84     20000
weighted avg       0.96      0.96      0.96     20000
```

Flask interface:



Github link:
https://github.com/taimurkhaneth